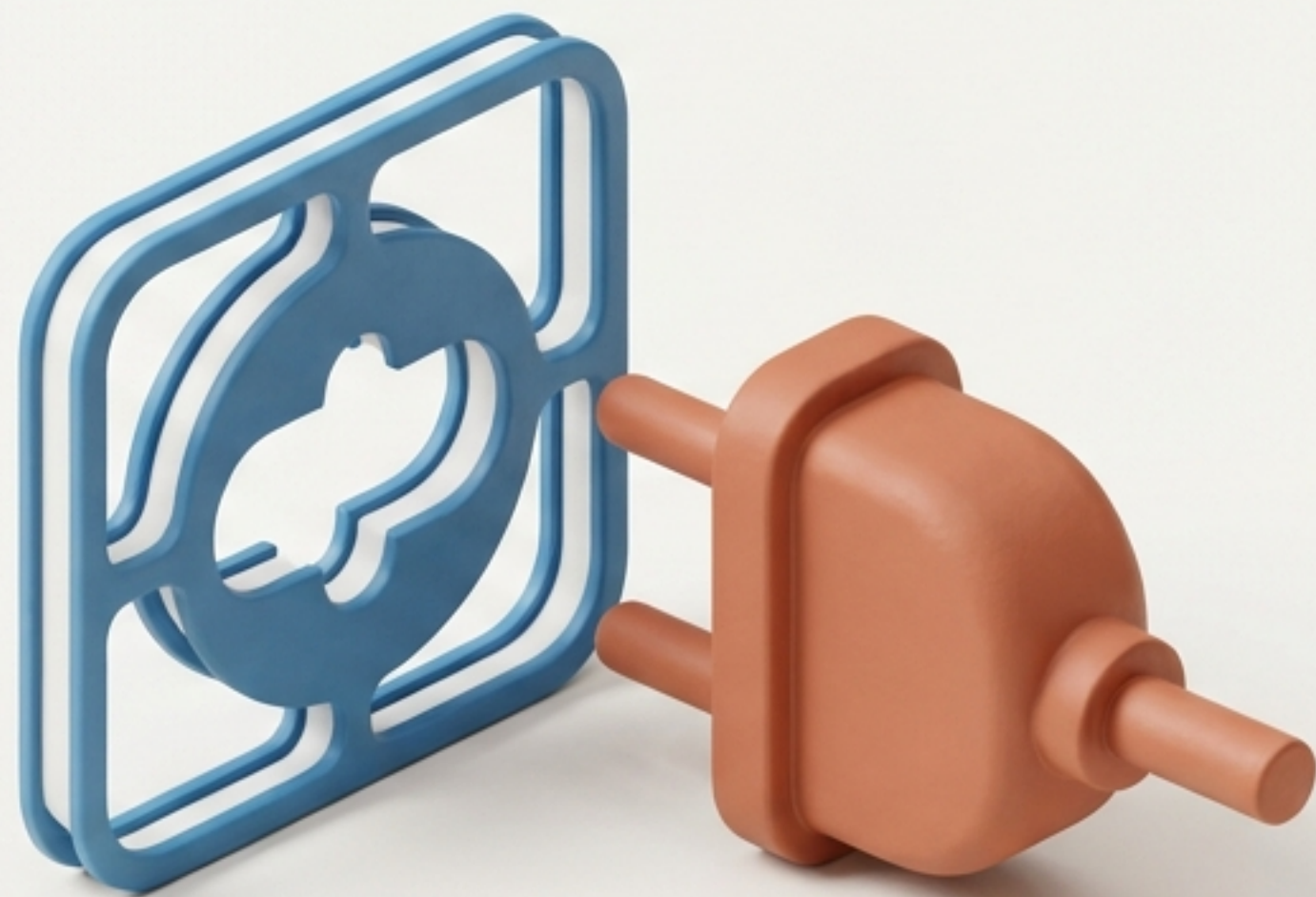




C# 인터페이스 완벽 가이드

에러 방지부터 다중 상속, 다형성까지 아키텍처의 핵심 이해하기

```

14 static void Main(string[] args)
15 {
16     List<Product> list = new List<Product>()
17     {
18         new Product() { Name = "고구마", Price = 1500 },
19         new Product() { Name = "사과", Price = 1000 },
20         new Product() { Name = "바나나", Price = 1200 },
21         new Product() { Name = "배", Price = 800 },
22     };
23     list.Sort();
24
25     foreach (var item in list)
26     {
27         Console.WriteLine(item);
28     }
29 }
30 }

```

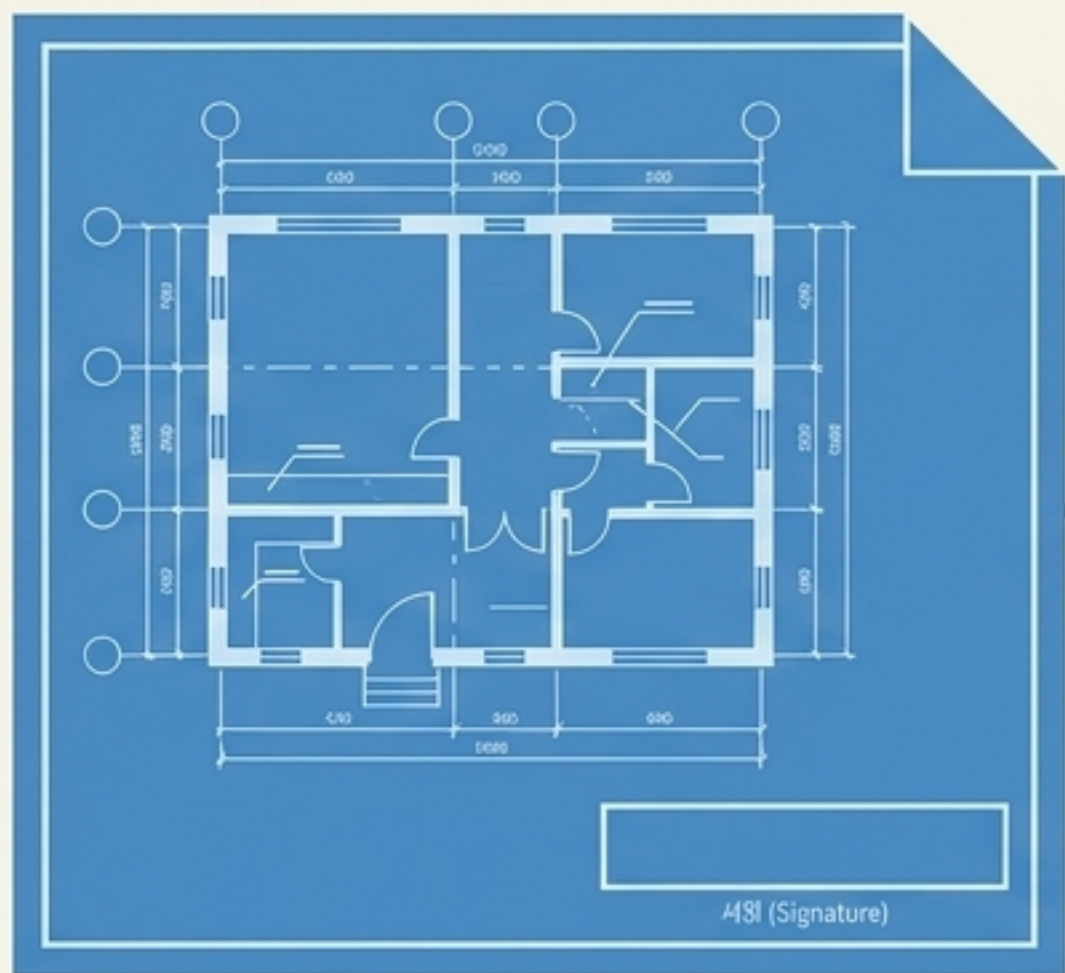
요로 특업

InvalidOperationException: 배열의 두 요소를 비교하지 못했습니다.

이름과 가격을 가진 Product 객체들,
왜 Sort() 메서드로 정렬할 수 없을까요?

컴퓨터는 어떤 기준(이름? 가격?)으로
정렬해야 할지 스스로 판단하지 못합니다.
명확한 기준을 알려줄 '규약'이 필요합니다.

인터페이스는 ‘엄격한 규약’입니다



의무 이행



정의: 클래스를 만들 때 반드시 따라야 하는
메서드나 속성의 목록.

특징: 실체가 없으며 내부 구현(로직)을 가질 수
없습니다. 오직 ‘규칙’만 존재하여 개발자의
실수를 원천 방지하는 안전장치 역할을 합니다.

IComparable로 정렬 기준 세우기

STEP 1

```
class Product : IComparable
{
    public string Name { get; set; }
    public int Price { get; set; }

    public override string ToString()
    {
        return Name + " : " + Price + "원";
    }
}
```

1단계: Product : IComparable 선언으로 C# 기본 제공 정렬 규약을 상속받습니다.

STEP 2

```
public int CompareTo(object obj)
{
    return this.Price.CompareTo((obj as Product).Price);
}
```

2단계: CompareTo() 메서드를 구현하여 '가격(Price)' 기준 비교 로직을 작성합니다.

리소스 정리의 마법, IDisposable

목적: 다 쓴 자원(메모리, 파일 등)을 안전하고 확실하게 해제하기 위한 규약입니다.

```
01 static void Main(string[] args)
02 {
03     using (Dummy dummy = new Dummy())
04     {
05
06     }
07 }
```



작동 원리: IDisposable을 구현한 클래스는 using 블록을 벗어나는 순간, 시스템이 알아서 Dispose() 메서드를 자동 호출합니다.

```
public void Dispose()
{
    Console.WriteLine("Dispose() 메서드를 호출합니다.");
}
```

IComparable과 IDisposable의 비교

구분	IComparable	IDisposable
핵심 목적	객체 간의 크기나 순서를 비교	사용이 완료된 시스템 자원을 안전하게 해제
필수 구현 메서드	CompareTo(object obj)	Dispose()
주요 작동 시점	List.Sort() 등의 정렬 메서드 실행 시	using (...) { } 블록이 끝날 때 자동 호출

설계도는 건물이 아닙니다!

```
Comparable comparable = new Comparable();
```



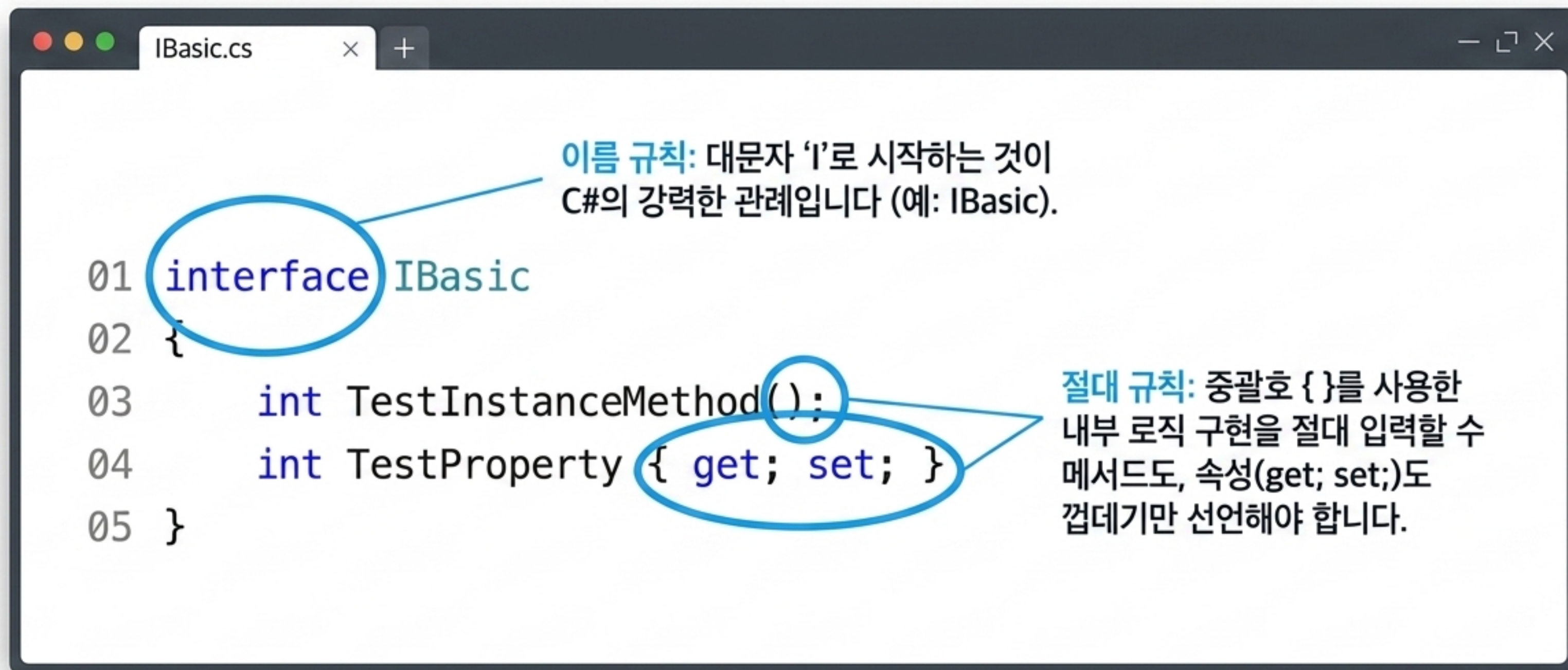
핵심 규칙:

인터페이스는 인터페이스는
인스턴스화할 수 없습니다.

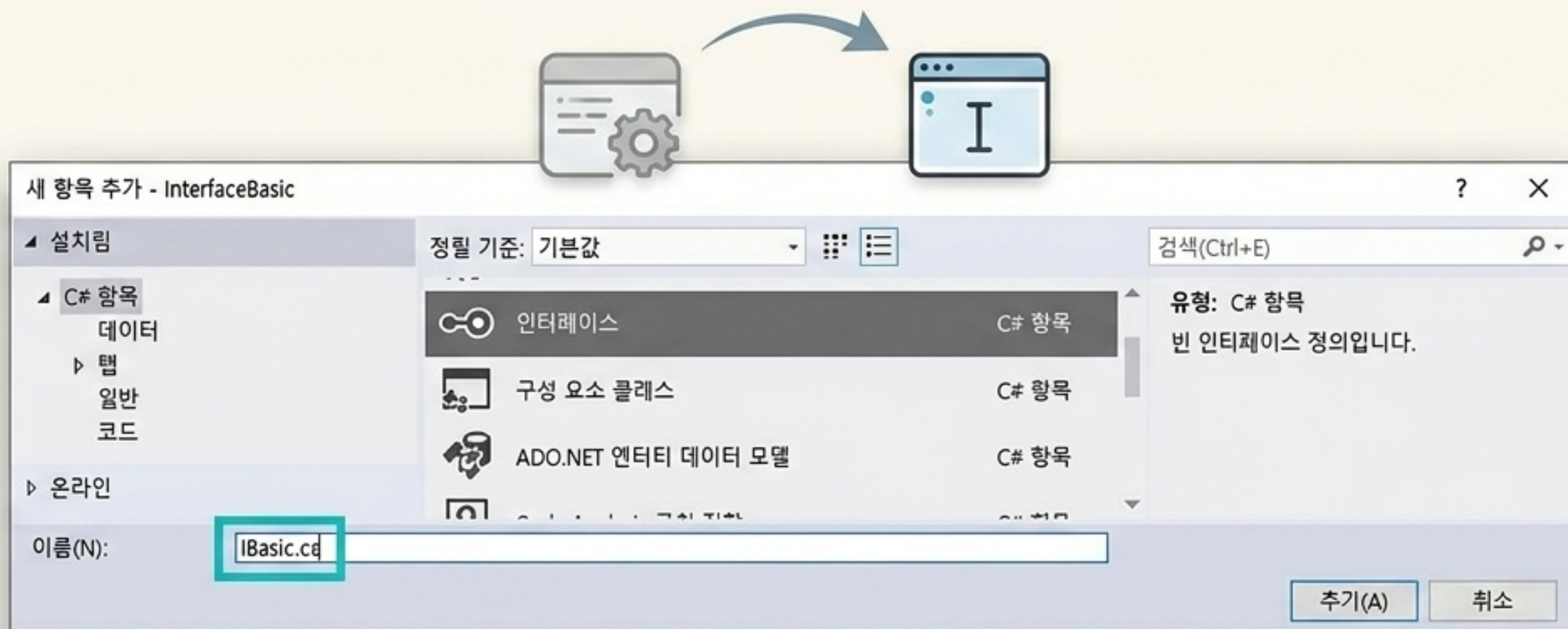
설명:

인터페이스는 실체가 없는 추상적인 '규칙'일 뿐입니다.
따라서 new 키워드를 사용하여 메모리에 직접 객체(인스턴스)를 생성하는 것은 문법적으로 절대 불가능합니다.

나만의 인터페이스 생성하기

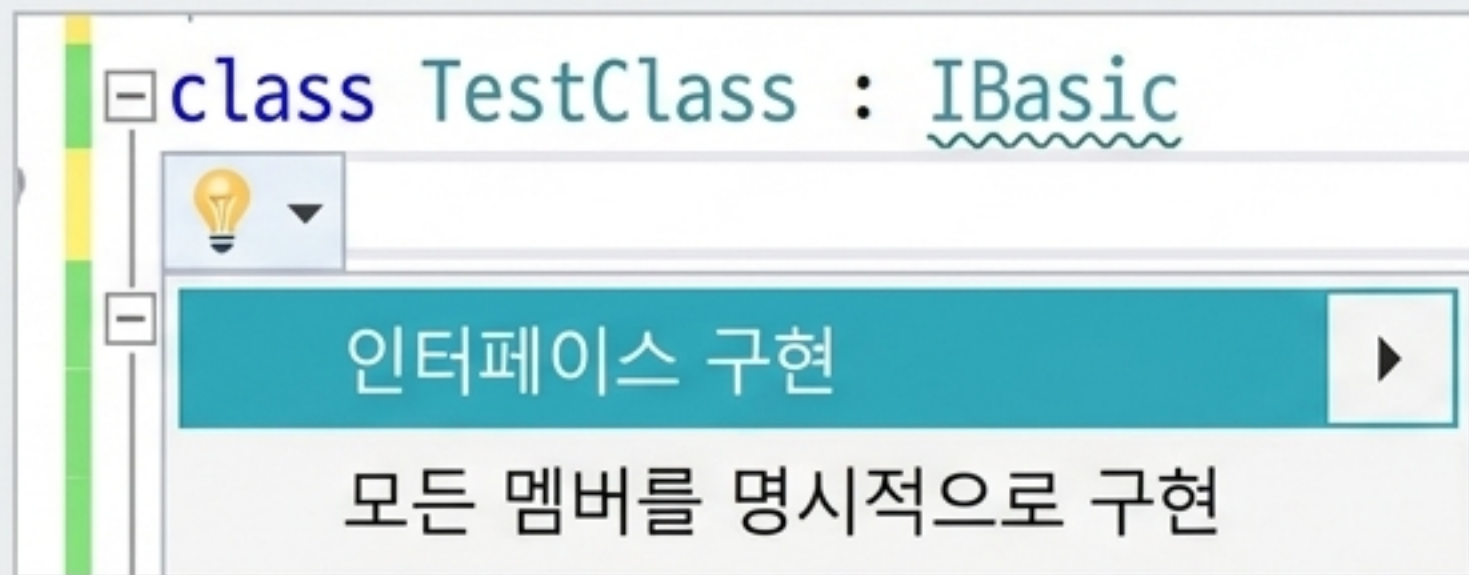


IDE Best Practice - 스마트 파일 생성



Pro Tip: 번거롭게 템플릿을 찾을 필요 없습니다. 새 파일 추가 시 이름을 IBasic.cs처럼 [대문자 I + 대문자]로 시작하게 입력하면, Visual Studio가 이를 스마트하게 인식하여 자동으로 인터페이스 템플릿을 완성해 줍니다.

상속과 뼈대 자동 완성



Action:

클래스명 뒤에 : IBasic을 입력하고 단축키 [Ctrl] + [.]를 눌러 [인터페이스 구현]을 클릭합니다.

Result:

타이핑할 필요 없이, 인터페이스에 정의된 모든 멤버의 뼈대가 클래스 내부에 즉시 자동 생성됩니다.

계약의 완성 (Fleshing out the Contract)

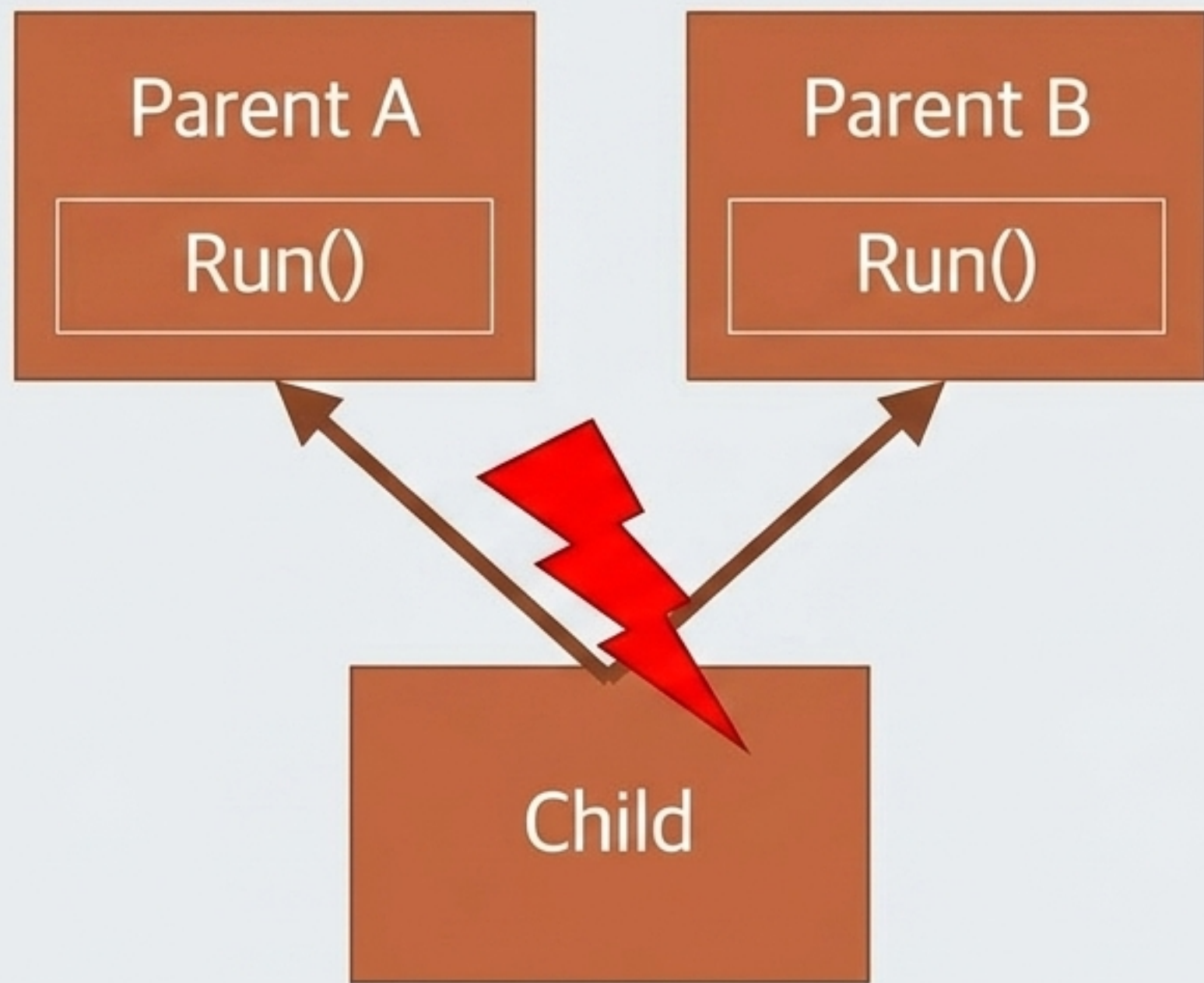
```
01 class TestClass : IBasic
02 {
03     public int TestInstanceMethod()
04     {
05         throw new NotImplementedException();
06     }
07
08     public int TestProperty
09     {
10         get => throw new NotImplementedException();
11         set => throw new NotImplementedException();
12     }
13 }
```

이 부분을 실제 로직으로 교체하세요

설명: 자동 생성된 코드는 방치하면 '아직 구현되지 않음(NotImplementedException)' 예외를 던지며 프로그램을 중단시킵니다.

의무: 개발자는 반드시 이 부분을 지우고, 인터페이스의 의도에 맞는 실제 동작 로직을 직접 채워 넣어야만 완벽한 컴파일이 가능합니다.

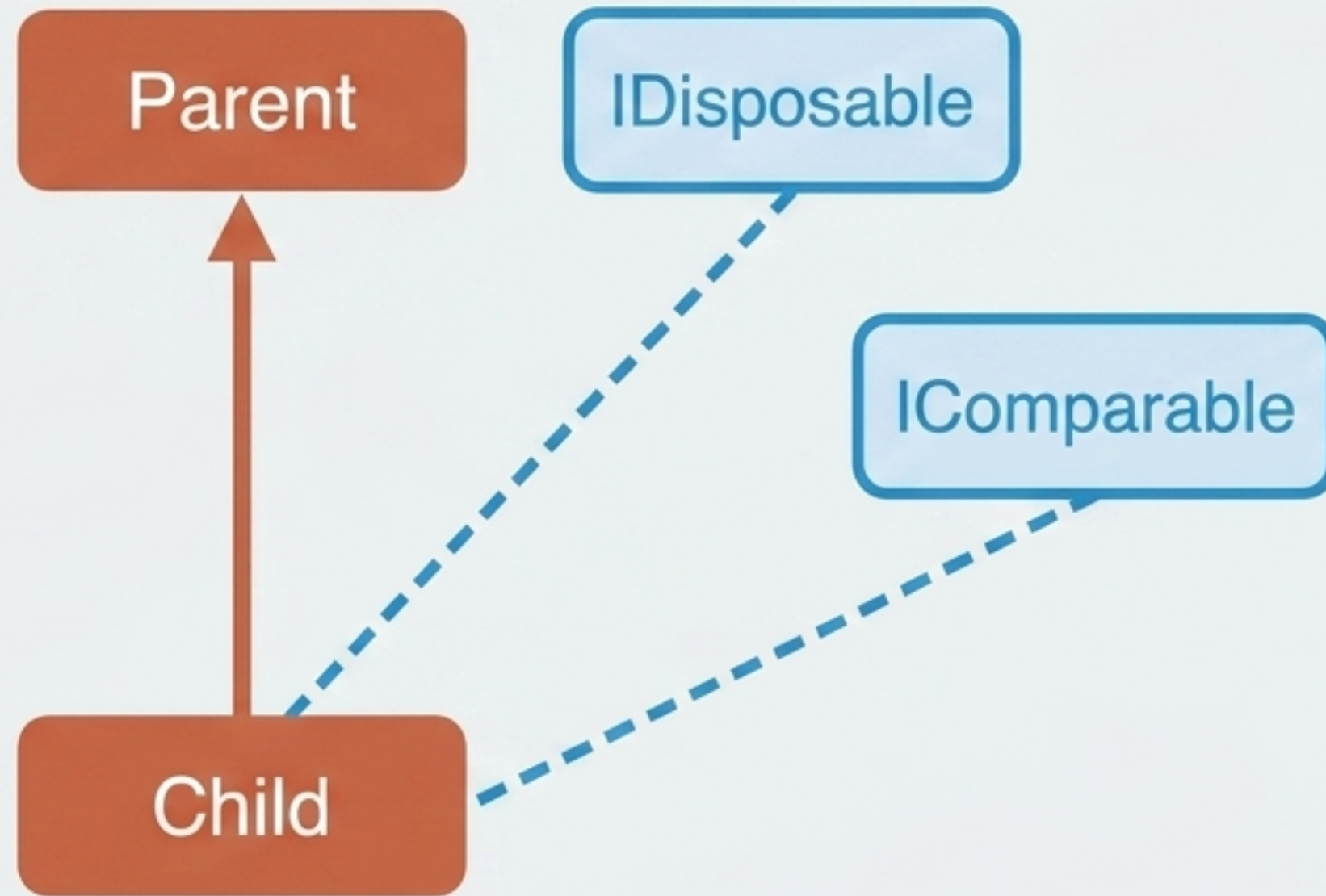
클래스 다중 상속의 치명적 한계



한계점: 과거 C++ 등에서는 클래스 다중 상속을 허용했으나, 부모들이 동일한 이름의 메서드를 가질 경우 발생하는 '모호성(Ambiguity)' 문제로 인해 아키텍처가 쉽게 붕괴되었습니다.

해결: 이 때문에 C#이나 Java 같은 모던 언어는 클래스의 다중 상속을 원천적으로 금지(단일 상속만 허용)합니다.

인터페이스, 한계를 넘어서다



원리: 인터페이스는 내부 로직(실체)이 없기 때문에 여러 개를 상속받아도 이름 충돌이나 모호성이 발생하지 않습니다.

결론: 1개의 메인 부모 클래스와 'N개'의 인터페이스 규약을 동시에 상속받아, 충돌 없이 강력하고 확장성 높은 클래스를 설계할 수 있습니다.

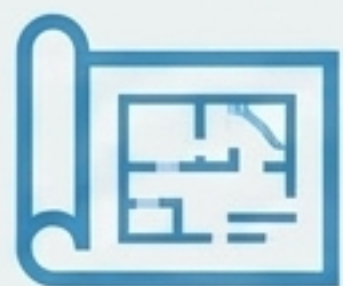
다형성(Polymorphism) - 다중 페르소나



설명: 다형성이란 하나의 객체가 상황에 따라 다양한 형태(Type)로 변신하여 취급될 수 있는 능력입니다.

핵심: N개의 인터페이스를 구현한 클래스는 N개의 '페르소나'를 갖게 되어, 각 규약이 요구되는 어떤 상황에서도 유연하고 독립적으로 동작할 수 있습니다.

[요약 종합: C# 인터페이스의 4가지 본질]



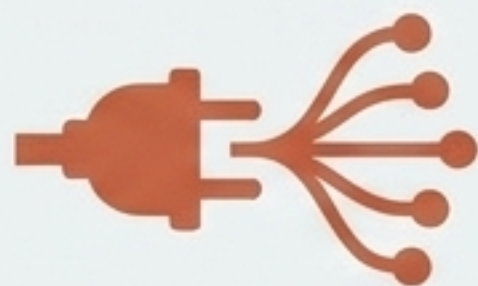
1. 엄격한 규약 (Contract)

클래스가 반드시 구현해야 하는
메서드/속성의 마스터 설계도.



2. 실체 없음 (No Implementation)

인스턴스화(new) 불가, 내부
로직 작성 불가.



3. 무한한 확장 (Multiple Inheritance)

클래스 단일 상속의 한계를 극복하는
안전한 다중 상속 지원.



4. 유연한 아키텍처 (Polymorphism)

하나의 객체가 다수의 페르소나를 갖는
다형성 달성.